

ELCT918 – Lab 2 Report

Comparative Study of Classical CNN Architectures — LeNet, AlexNet, and VGG16

Team members

Name	ID
Nada Yehia Elsayed	S2701048
Sama Samy Mohamed	S2701051

Instructors:

Dr.Eman Azab

Eng. Nour Elshahawy

1. Problem Summary

The objective of this lab is to implement and compare three classical convolutional neural network architectures: **LeNet-5**, **AlexNet**, and **VGG16**. These architectures represent different stages in the development of convolutional neural networks, with significant differences in depth, filter size, number of feature maps, number of parameters, and computational requirements.

The three networks were implemented using **TensorFlow/Keras** and adapted to work with the **32×32 RGB images** used by CIFAR-10 and CIFAR-100. All models are trained from scratch without using pretrained weights.

The models are evaluated on two datasets:

- **CIFAR-10**: 10 image classes.
- **CIFAR-100**: 100 image classes.

Therefore, six experiments are performed:

1. LeNet-5 on CIFAR-10
2. AlexNet on CIFAR-10
3. VGG16 on CIFAR-10
4. LeNet-5 on CIFAR-100
5. AlexNet on CIFAR-100
6. VGG16 on CIFAR-100

The comparison considers both the architectural properties of the networks and their behavior during training and testing. The assignment requires comparing layer structures, trainable parameters, accuracy, accuracy drop, training/validation curves, and training time per epoch.

2. Framework and Implementation

TensorFlow/Keras was selected as the implementation framework for all three architectures. Using the same framework provides a consistent environment for comparing the models.

The networks were implemented explicitly using Keras layers such as:

- Conv2D
- MaxPooling2D
- Flatten
- Dense
- a custom trainable subsampling layer for LeNet-5

No pretrained model weights were used. All models are initialized and trained from scratch as required.

The input shape for both CIFAR datasets is:

$32 \times 32 \times 3$

where the three channels correspond to the RGB color channels.

The final classification layer was adapted according to the dataset:

- CIFAR-10 $\rightarrow$ 10 output classes
- CIFAR-100 $\rightarrow$ 100 output classes

3. CIFAR Adaptations

The original networks were designed for different input sizes and classification problems. Therefore, some modifications are necessary to apply them to CIFAR-10 and CIFAR-100.

3.1 LeNet-5 Adaptation

The original LeNet-5 was designed for grayscale handwritten-digit images. The CIFAR datasets instead contain RGB images with three input channels.

Therefore, the first convolutional layer receives three input channels:

$32 \times 32 \times 3$ instead of a single grayscale channel.

The main LeNet-5 structure is preserved:

```
Input
↓
C1: 5×5 convolution, 6 feature maps
↓
S2: 2×2 subsampling
↓
C3: 5×5 convolution, 16 feature maps
↓
S4: 2×2 subsampling
↓
C5: 5×5 convolution, 120 feature maps
↓
F6: 84 neurons
↓
Output:Dense(10 or 100)
```

The S2 and S4 layers were implemented as trainable subsampling layers. Each feature map has one trainable coefficient and one trainable bias. The four values in each non-overlapping 2×2 region are summed, followed by multiplication by the trainable coefficient, addition of the trainable bias, and a sigmoid activation. This follows the subsampling operation described in the original LeNet-5 architecture.

3.2 AlexNet Adaptation

The original AlexNet was designed for much larger images than CIFAR's 32×32 images. In particular, its first convolution uses a large filter and a large stride.

Using the original stride directly on a 32×32 image would reduce the spatial dimensions too aggressively. Therefore, the first convolution was changed to:

11×11,stride=1

instead of using the original large stride.

The main five-convolutional-layer and three-fully-connected-layer structure was retained.

The resulting structure is:

```
Input
↓
11×11 Conv, 96 filters
↓
3×3 Max Pool
↓
5×5 Conv, 256 filters
↓
3×3 Max Pool
↓
3×3 Conv, 384 filters
↓
3×3 Conv, 384 filters
↓
3×3 Conv, 256 filters
↓
3×3 Max Pool
↓
Flatten
↓
4096 FC
↓
4096 FC
↓
Output
```

The final layer was changed from the original ImageNet classification output to either 10 or 100 classes.

3.3 VGG16 Adaptation

VGG16 was originally designed for much larger RGB images. For this experiment, the input was changed to:

32×32×3

while preserving the characteristic VGG16 structure of **13 convolutional layers and 3 fully connected layers**.

The network uses small:

3×3

convolutional filters throughout the convolutional blocks.

The five VGG blocks are:

Block 1: Conv2D(64, 3×3), Conv2D(64, 3×3), MaxPooling2D(2×2)

Block 2: Conv2D(128, 3×3), Conv2D(128, 3×3), MaxPooling2D(2×2)

Block 3: Conv2D(256, 3×3), Conv2D(256, 3×3), Conv2D(256, 3×3), MaxPooling2D(2×2)

Block 4: Conv2D(512, 3×3), Conv2D(512, 3×3), Conv2D(512, 3×3), MaxPooling2D(2×2)

Block 5: Conv2D(512, 3×3), Conv2D(512, 3×3), Conv2D(512, 3×3), MaxPooling2D(2×2)

For the 32×32 CIFAR input, the spatial dimensions become:

32→16→8→4→2→1

after the five pooling operations.

The final classification layer was adapted to 10 or 100 classes.

4. Architectural Comparison

4.1 LeNet-5

The implemented LeNet-5 architecture is:

Layer	Type	Configuration	Output Shape	Parameters
1	Convolution	6 filters, 5×5, stride 1	28×28×6	456
2	Trainable Subsampling	2×2, stride 2	14×14×6	12
3	Convolution	16 filters, 5×5, stride 1	10×10×16	2,416
4	Trainable Subsampling	2×2, stride 2	5×5×16	32
5	Convolution	120 filters, 5×5, stride 1	1×1×120	48,120
6	Flatten	—	120	0
7	Fully Connected	84 units	84	10,164
8	Output	10 units	10	850

For CIFAR-10, the total number of trainable parameters is: 62,050

4.2 AlexNet

Layer	Type	Configuration	Output Shape	Parameters
1	Convolution	96 filters, 11×11, stride 1	32×32×96	34,944
2	Max Pooling	3×3, stride 2	15×15×96	0
3	Convolution	256 filters, 5×5, stride 1	15×15×256	614,656
4	Max Pooling	3×3, stride 2	7×7×256	0
5	Convolution	384 filters, 3×3, stride 1	7×7×384	885,120
6	Convolution	384 filters, 3×3, stride 1	7×7×384	1,327,488
7	Convolution	256 filters, 3×3, stride 1	7×7×256	884,992
8	Max Pooling	3×3, stride 2	3×3×256	0
9	Flatten	—	2304	0
10	Fully Connected	4096 units	4096	9,441,280
11	Fully Connected	4096 units	4096	16,781,312
12	Output	10 units	10	40,970

For CIFAR-10, the total number of trainable parameters is: 30,010,762

4.3 VGG16

Layer	Type	Configuration	Output Shape	Parameters
1	Convolution	64 filters, 3×3	32×32×64	1,792
2	Convolution	64 filters, 3×3	32×32×64	36,928
3	Max Pooling	2×2, stride 2	16×16×64	0
4	Convolution	128 filters, 3×3	16×16×128	73,856
5	Convolution	128 filters, 3×3	16×16×128	147,584
6	Max Pooling	2×2, stride 2	8×8×128	0
7	Convolution	256 filters, 3×3	8×8×256	295,168
8	Convolution	256 filters, 3×3	8×8×256	590,080
9	Convolution	256 filters, 3×3	8×8×256	590,080
10	Max Pooling	2×2, stride 2	4×4×256	0
11	Convolution	512 filters, 3×3	4×4×512	1,180,160
12	Convolution	512 filters, 3×3	4×4×512	2,359,808
13	Convolution	512 filters, 3×3	4×4×512	2,359,808
14	Max Pooling	2×2, stride 2	2×2×512	0
15	Convolution	512 filters, 3×3	2×2×512	2,359,808
16	Convolution	512 filters, 3×3	2×2×512	2,359,808
17	Convolution	512 filters, 3×3	2×2×512	2,359,808
18	Max Pooling	2×2, stride 2	1×1×512	0
19	Flatten	—	512	0
20	Fully Connected	4096 units	4096	2,101,248
21	Fully Connected	4096 units	4096	16,781,312
22	Output	10 units	10	40,970

For CIFAR-10, the total number of trainable parameters is: 33,638,218

For CIFAR-100, only the final classification layer changes from 10 to 100 outputs, so the total parameter count increases accordingly.

5. Overall Architectural Comparison

Model	Convolutional Layers	FC Layers	Total Weighted Layers	Trainable Parameters*
LeNet-5	3	2	5	62,006
AlexNet	5	3	8	30,010,762
VGG16	13	3	16	33,638,218

Values shown are for the CIFAR-10 version with 10 output classes.

The three architectures show a clear progression in depth and model capacity.

LeNet-5 is the shallowest and smallest model. It uses relatively large 5×5 filters and only a small number of feature maps.

AlexNet is considerably deeper and wider than LeNet-5. It uses five convolutional layers and large fully connected layers containing millions of parameters.

VGG16 increases depth significantly by using 13 convolutional layers. Instead of relying mainly on large convolutional filters, VGG16 repeatedly applies small 3×3 filters.

6. Filter Size and Receptive Field

One of the main differences between the architectures is the choice of convolutional filter size.

LeNet-5 uses 5×5 convolutional filters, while AlexNet starts with a much larger 11×11 filter and then uses 5×5 and 3×3 filters.

VGG16 primarily uses 3×3 filters.

Stacking multiple 3×3 convolutional layers allows the network to build a larger effective receptive field while inserting nonlinear transformations between the convolutional operations.

For example, two consecutive 3×3 convolutions have an effective receptive field of:

$$3+3-1=5$$

while three consecutive 3×3 convolutions have an effective receptive field of:

$$3+3+3-2=7$$

Therefore, VGG16 can obtain larger effective receptive fields using several small filters rather than directly using large filters.

7. Depth, Width, and Parameter Count

Increasing the number of layers allows a CNN to learn increasingly complex hierarchical representations.

LeNet-5 has relatively few layers and feature maps. It is therefore computationally smaller and has much lower parameter storage requirements.

AlexNet increases both the number of convolutional filters and the size of the fully connected layers. Its parameter count increases dramatically to approximately 30 million.

VGG16 goes even deeper, using 13 convolutional layers. Its parameter count in the current CIFAR adaptation is approximately 33.6 million.

The difference between LeNet-5 and VGG16 is particularly large:

$$33,638,218 / 62,050 \approx 542$$

Therefore, the current VGG16 implementation has approximately **542 times more trainable parameters** than the current LeNet-5 implementation.

A larger parameter count provides greater model capacity, but it also increases memory requirements, computational cost, and the possibility of overfitting.

8. Training Setup

To make the architectural comparison meaningful, the same training configuration is used for the three models within each dataset, as required by the assignment.

The training configuration used in the implementation is:

Setting	Value
Framework	TensorFlow/Keras
Epochs	6
Batch Size	128
Optimizer	Adam
Learning Rate	0.001
Loss Function	Sparse Categorical Crossentropy
Training	From scratch
Input Size	32×32×3

For CIFAR-10, the output layer contains 10 neurons.

For CIFAR-100, the output layer contains 100 neurons.

The models use sparse categorical crossentropy because the class labels are represented as integer class indices rather than one-hot encoded vectors.

Training time is measured at the end of every epoch. The average epoch time is used as a rough indication of computational cost.

9. Evaluation Metrics

The following metrics are recorded for every model:

Top-1 Accuracy

Top-1 accuracy measures the percentage of test images for which the model's highest-probability prediction is the correct class.

Top-5 Accuracy

Top-5 accuracy is particularly useful for CIFAR-100 because there are 100 possible classes. A prediction is considered correct if the true class appears among the model's five highest-probability predictions.

Accuracy Drop

The assignment defines accuracy drop as:

$$\text{Accuracy Drop} = 100 - \text{Test Accuracy}$$

where test accuracy is expressed as a percentage.

10. Experimental Results

The following table will contain the final results after training all six models.

Model	Dataset	Test Top-1 Accuracy	Top-5 Accuracy	Accuracy Drop	Average Epoch Time
LeNet-5	CIFAR-10	50 %	93%	50%	26.25s
AlexNet	CIFAR-10	60%	95.7%	40%	2977.95s =49.6min
VGG16	CIFAR-10	9.85 %	4.99%	90.15%	1hour
LeNet-5	CIFAR-100	21 %	49%	79%	29.67s
AlexNet	CIFAR-100	18.86%	44.7%	81.14%	2597.47s =43.30min
VGG16	CIFAR-100	0.89 %	4.59%	99.11 %	2hours

11. Training and Validation Curves

For each model, training and validation curves are generated across all epochs.

These curves are useful for understanding not only the final accuracy but also the behavior of each model during training.

A model whose training accuracy continues to increase while validation accuracy stops improving or decreases may show signs of overfitting.

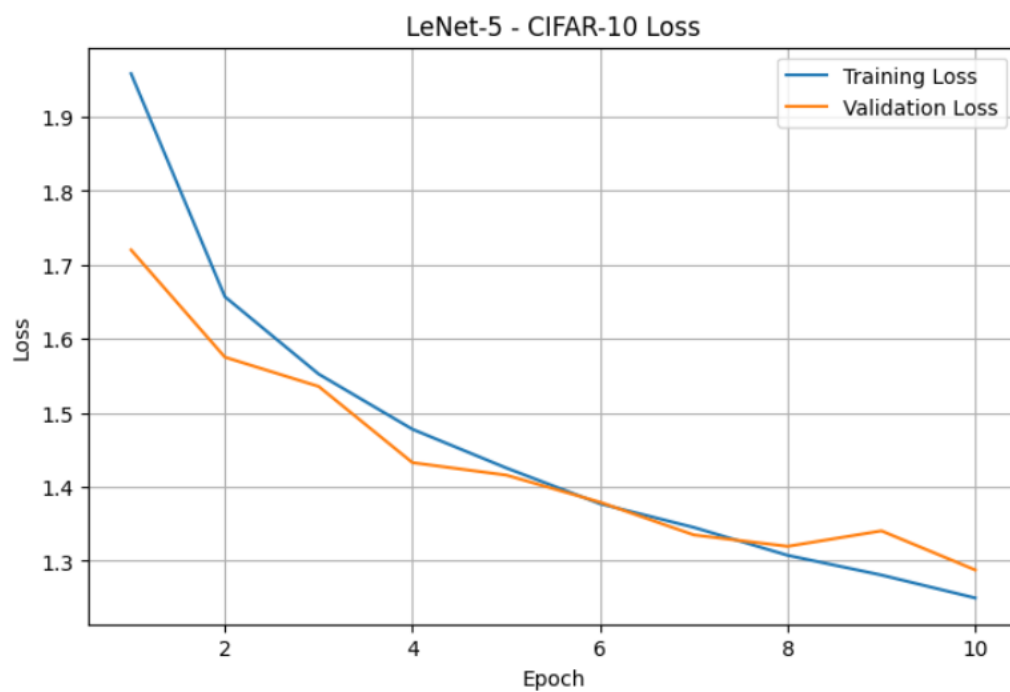
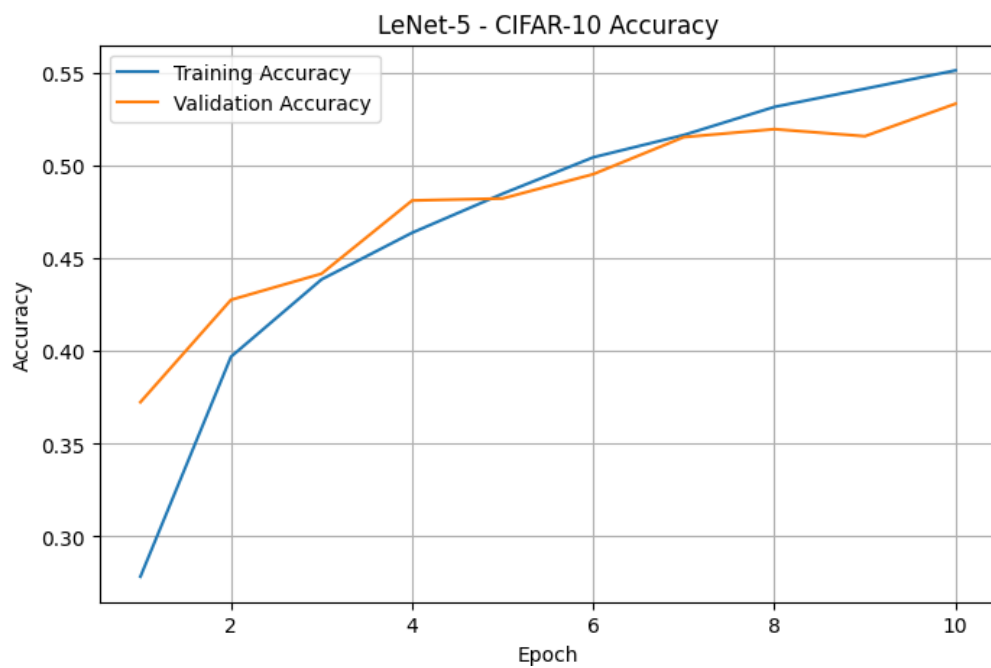


Figure 1: LeNet-5 on CIFAR-10 training curves

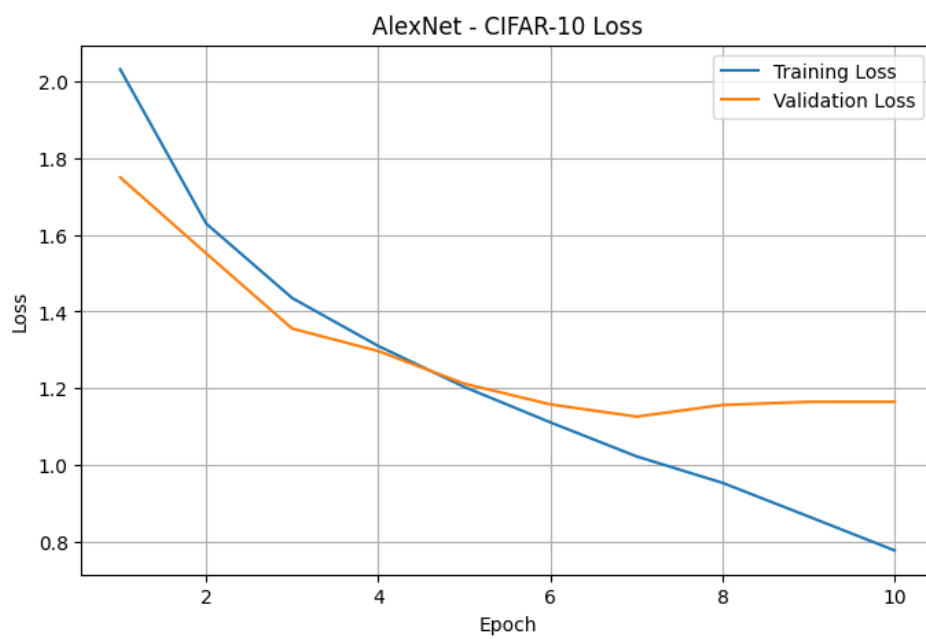
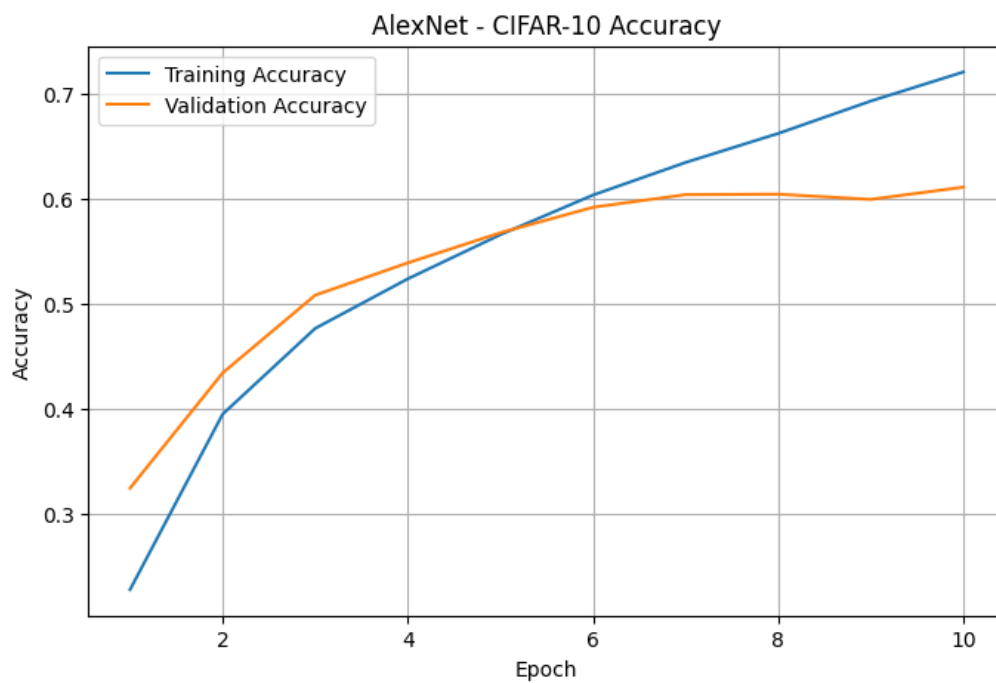


Figure 2: AlexNet on CIFAR-10 training curves

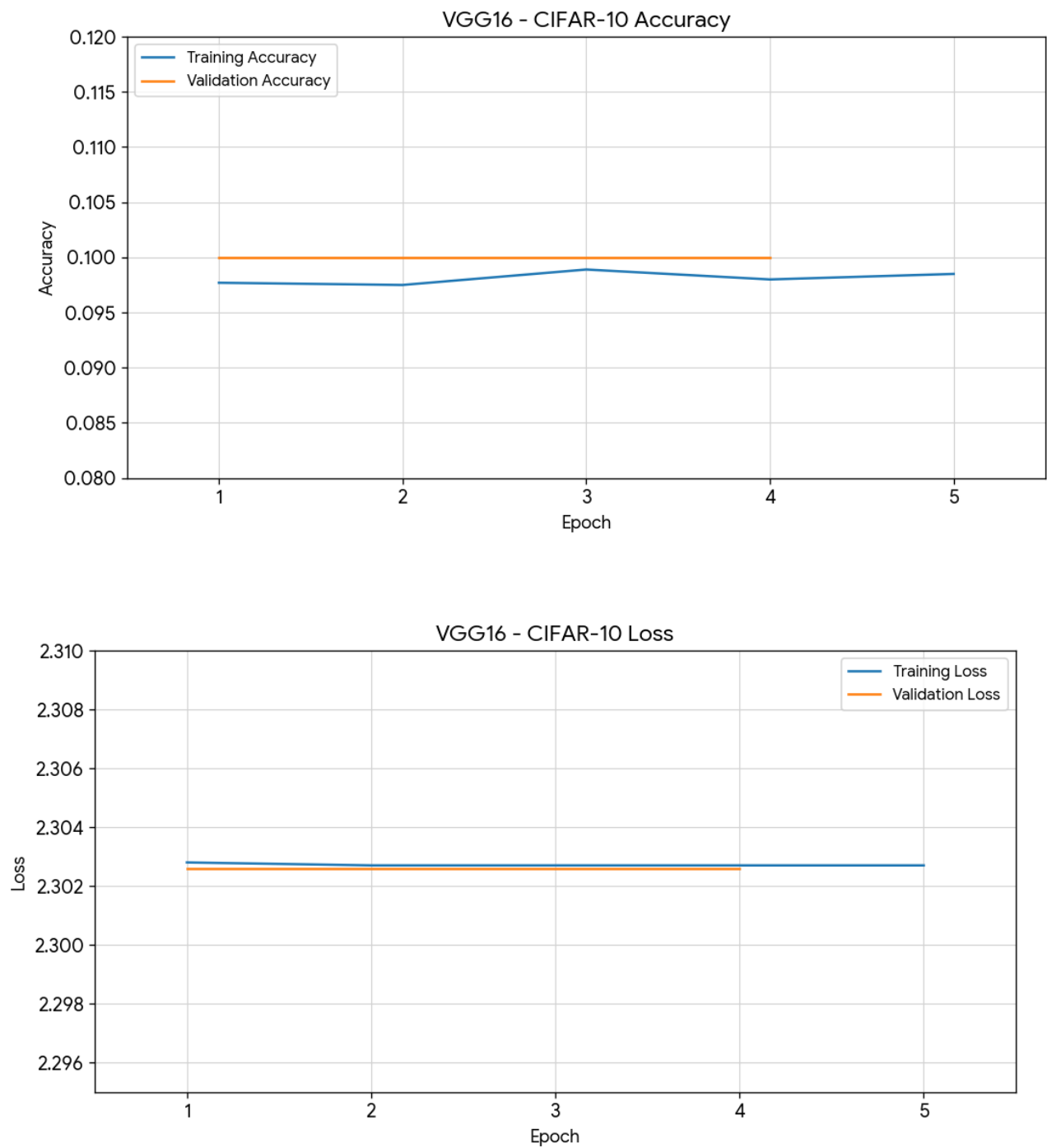


Figure 3: VGG16 on CIFAR-10 training curves

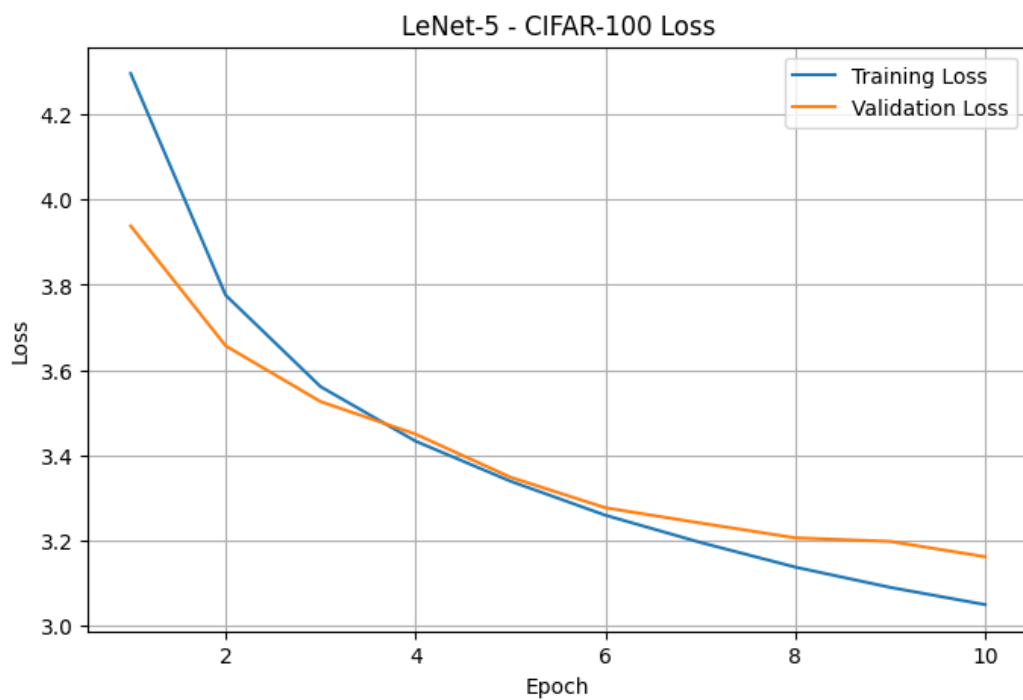
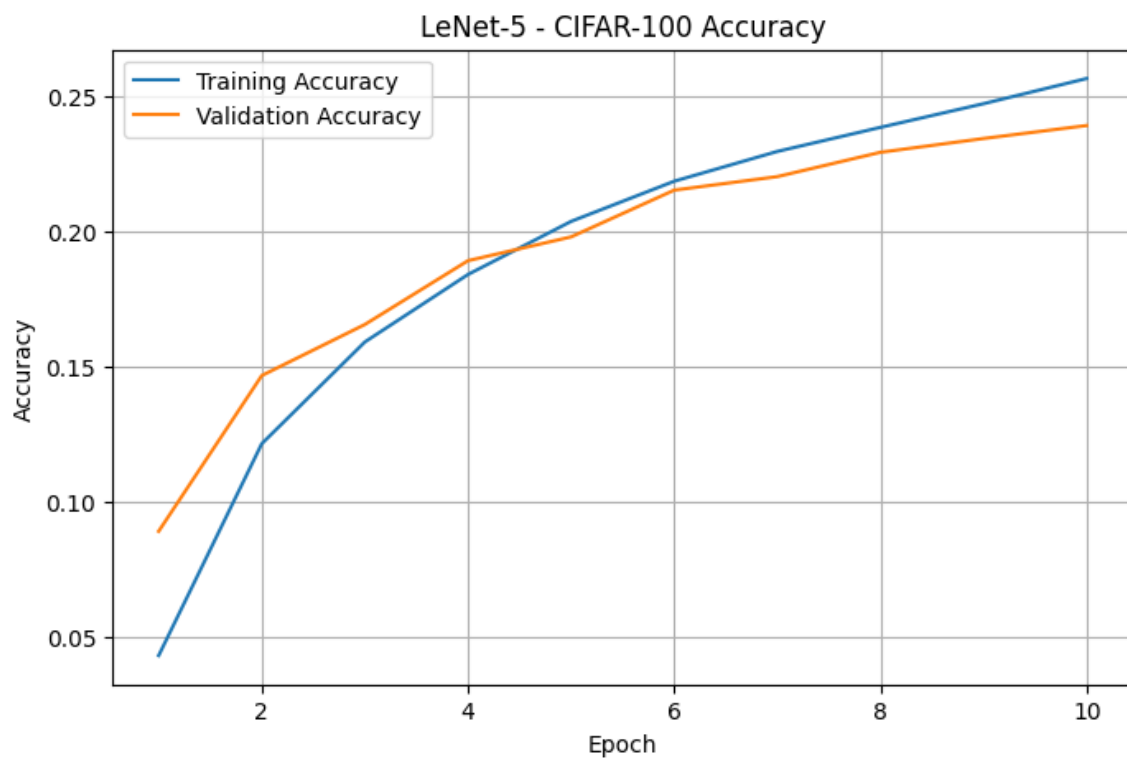


Figure 4: LeNet-5 on CIFAR-100 training curves

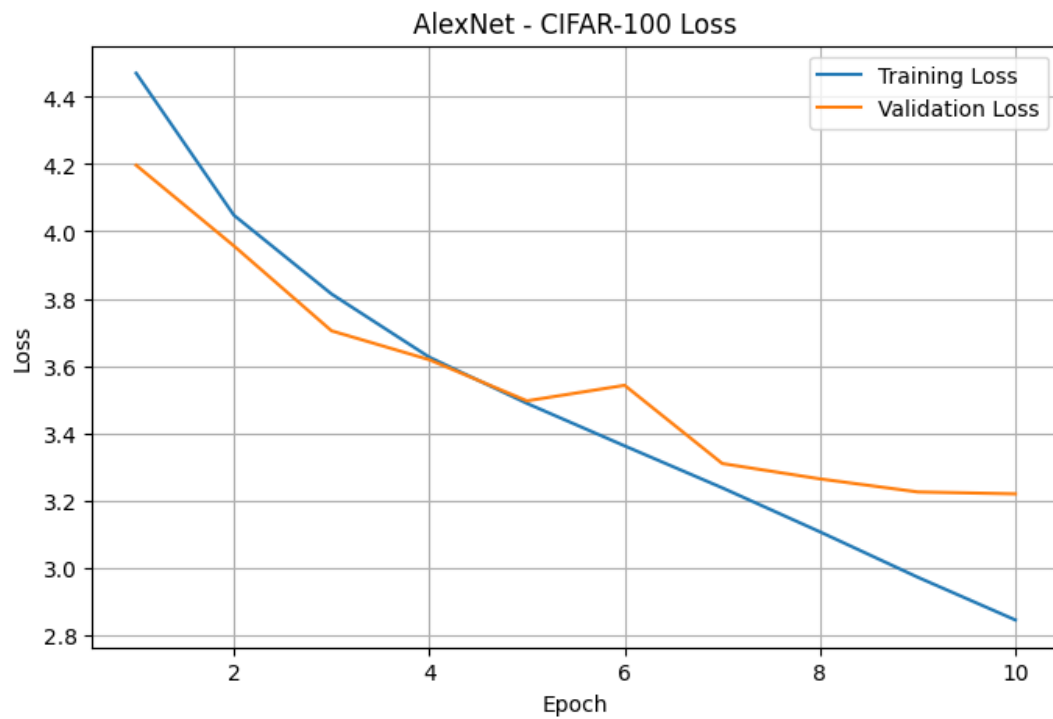
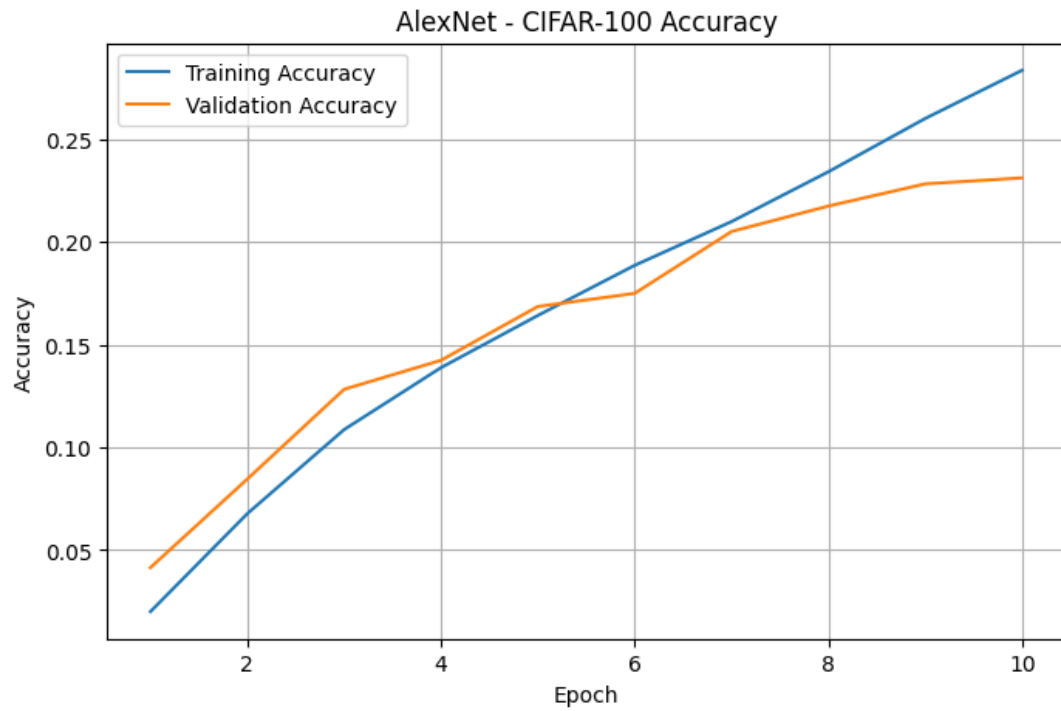


Figure 5: AlexNet on CIFAR-100 training curves

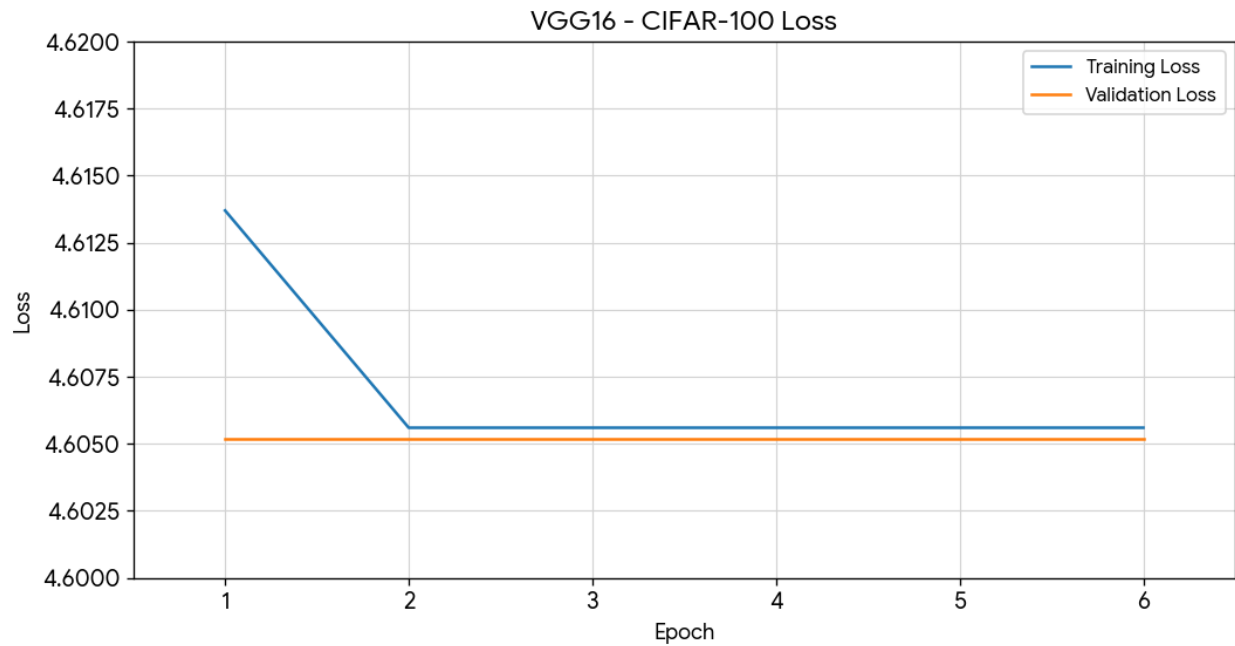
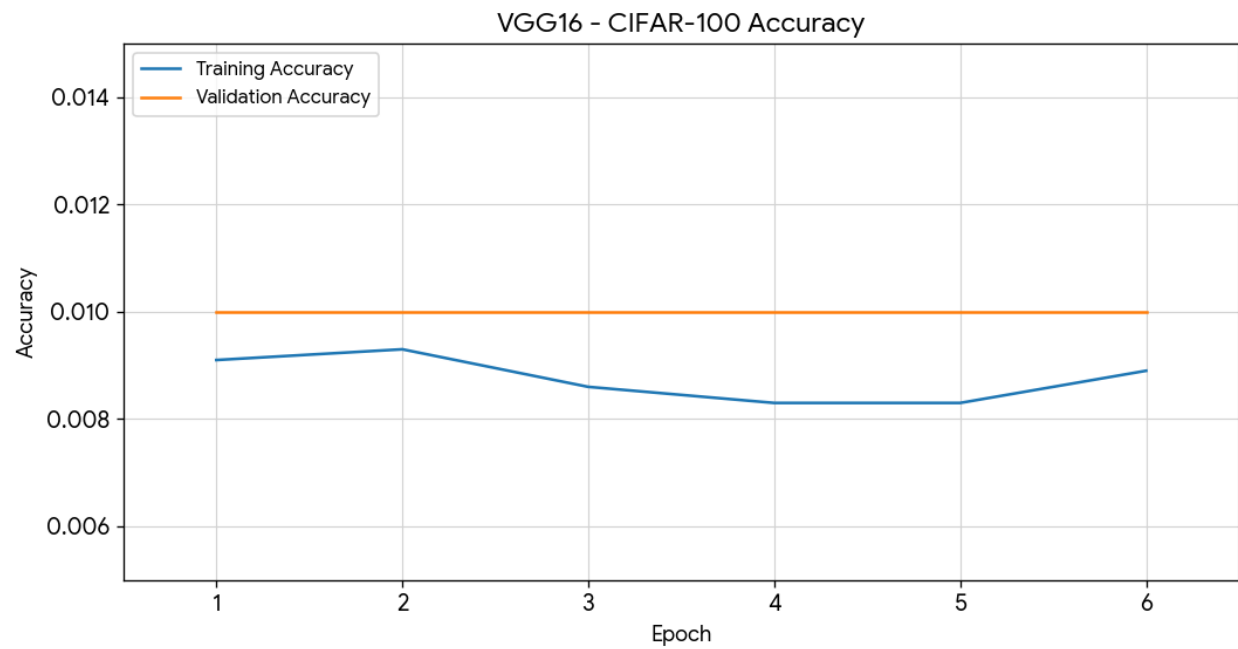


Figure 6: VGG16 on CIFAR-100 training curves

12. Comparative Discussion

12.1 CIFAR-10 versus CIFAR-100

All networks perform better on CIFAR-10 because it has only 10 classes. CIFAR-100 is more difficult because the models must distinguish between 100 classes, which is particularly challenging for shallower or narrower networks such as LeNet-5 to learn sufficiently.

12.2 Effect of Increasing Network Depth

LeNet-5, AlexNet, and VGG16 provide three substantially different levels of network depth.

LeNet-5 contains only three convolutional layers, whereas AlexNet contains five and VGG16 contains thirteen.

However, VGG16 does not show an improvement proportional to its much greater depth and parameter count. In fact, on CIFAR-100 it showed almost no learning during the first six epochs. This suggests that simply increasing model size does not guarantee better performance under the same training setup and limited training time.

12.4 Overfitting

- **AlexNet on CIFAR-10 (Most Overfitting):** After epoch 5, the training accuracy keeps rising up to 72%. However, the validation accuracy completely stops growing at around 60%. That large, growing gap shows clear overfitting.
 - **AlexNet on CIFAR-100 (Mild Overfitting):** From epoch 7, training accuracy keeps going up to 28%, but validation accuracy gets stuck at 23%.
 - **LeNet-5 on CIFAR-10 & CIFAR-100 (Very Little Overfitting):** Both lines stay very close together from start to finish.
-

14. Conclusion

This lab implemented three classical CNN architectures—LeNet-5, AlexNet, and VGG16—using TensorFlow/Keras and adapted them to the 32×32 RGB input format of CIFAR-10 and CIFAR-100.

The architectural comparison demonstrates a clear progression from a small and shallow network to increasingly deeper and higher-capacity architectures.

LeNet-5 has:

62,006

trainable parameters.

AlexNet has:

30,010,762

trainable parameters.

VGG16 has:

33,638,218

trainable parameters.

The CIFAR-100 experiments additionally provide top-5 accuracy, allowing the behavior of the models to be examined on a more difficult 100-class classification problem.

Overall, the experiment demonstrates the trade-off between **network depth, width, parameter count, computational cost, and classification performance** across three important CNN architectures.

15. GitHub Repository.

<https://github.com/Nada44yehia/ELCT918.git>